

Database Management System
ECS 403

Lecture 10,11: Inner & outer join

By

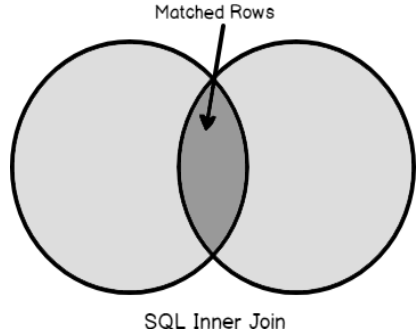
Ms. Bapita Roy
Head of the department
Electronics & Computer Science Engineering
Guru Nanak Institute of Technology

Contact no: 9231893138, Mail Id: head_ecs.gnit@jisgroup.org

Definition of SQL Inner Join

Inner Join clause in SQL Server creates a new table (not physical) by combining rows that have matching values in two or more tables. This join is based on a logical relationship (or a common field) between the tables and is used to retrieve data that appears in both tables.

Assume, we have two tables, Table A and Table B, that we would like to join using SQL Inner Join. The result of this join will be a new result set that returns matching rows in both these tables. The intersection part in black below shows the data retrieved using Inner Join in SQL Server.





Syntax of INNER JOIN

The syntax of INNER JOIN is:

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.column_name = table2.column_name;
```

Examples of Inner or Natural Join

SQL INNER JOIN

Table: Customers

customer_id	first_name
1	John
2	Robert
<u>3</u>	David
4	John
<u>5</u>	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	<u>3</u>
3	300	6
4	800	<u>5</u>
5	150	8



customer_id	first_name	amount
3	David	500
5	Betty	800

➤ Write the query language to get the final result after inner join of the given relations- Customers and Orders

The SQL INNER JOIN joins two tables based on a common column, and selects records that have matching values in these columns.

```
SELECT Customers.customer_id, Customers.first_name, Orders.amount  
FROM Customers  
INNER JOIN Orders  
ON Customers.customer_id = Orders.customer;
```

Input



Run SQL



Available Tables

```
SELECT Customers.customer_id, Customers.first_name, Orders.amount
FROM Customers
INNER JOIN Orders
ON Customers.customer_id = Orders.customer;
```

Output

customer_id	first_name	amount
3	David	500
5	Betty	800

Customers

customer_id	first_name
1	John
2	Robert
3	David
4	John
5	Betty

Orders

order_id	amount	customer
1	200	10
2	500	3
3	300	6
4	800	5
5	150	8

Example

Consider the following two tables

Table 1 – CUSTOMERS Table is as follows.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

- Write SQL to inner join these two tables:

Table 2 – ORDERS Table is as follows.

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

Now, let us join these two tables using the INNER JOIN as follows –

```
SQL> SELECT ID, NAME, AMOUNT, DATE  
FROM CUSTOMERS  
INNER JOIN ORDERS  
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

This would produce the following result.

ID	NAME	AMOUNT	DATE
3	kaushik	3000	2009-10-08 00:00:00
3	kaushik	1500	2009-10-08 00:00:00
2	Khilan	1560	2009-11-20 00:00:00
4	Chaitali	2060	2008-05-20 00:00:00



INNER JOIN With WHERE Clause

Here's an example of the INNER JOIN with the **WHERE** clause:

```
SELECT Customers.customer_id, Customers.first_name, Orders.amount  
FROM Customers  
INNER JOIN Orders  
ON Customers.customer_id = Orders.customer  
WHERE Orders.amount >= 500;
```

Input



Run SQL



Available Tables

```
SELECT Customers.customer_id, Customers.first_name, Orders.amount
FROM Customers
INNER JOIN Orders
ON Customers.customer_id = Orders.customer
WHERE Orders.amount >= 500;
```

Output

customer_id	first_name	amount
3	David	500
5	Betty	800

Customers

customer_id	first_name
1	John
2	Robert
3	David
4	John
5	Betty

Orders

order_id	amount	customer
1	200	10
2	500	3
3	300	6
4	800	5
5	150	8

Theta join

A theta join is a **join that links tables based on a relationship other than equality between two columns**. A theta join could use any operator other than the "equal" operator.

□ **Theta join / Conditional Join**

- It combines tuples from different relations provided they satisfy the **theta (θ) condition**.
- It is a **general case of join**. And it is **used** when we want to **join** two or more relation based on some **conditions**.
- The **join condition** is denoted by the symbol θ .
- It **uses** all kinds of **comparison operators** like $<, >, <=, >=, =, \neq$

□ **Notation: $A \bowtie_{\theta} B$**

Where θ is a **predicate/condition**. It can use any comparison operator ($<, >, <=, >=, =, \neq$)

□ **$A \bowtie_{\theta} B = \sigma_{\theta}(A \times B)$**

Example of Theta Join

S1

sid	name	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	35.0

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

S1 $\bowtie_{S1.sid < R1.sid}$ **R1**

Equivalent to : $\sigma_{S1.sid < R1.sid} (S1 \times R1)$

So, $A \bowtie_{\theta} B = \sigma_{\theta} (A \times B)$

Equi join

- When a theta join uses only equivalence (=) condition, it becomes a **Equi join**.
- **Equi join** is a **special case of theta (or conditional) join** where condition contains **equalities (=)**.
- **Notation:** $A \bowtie_{A.a_1 = B.b_1 \wedge \dots \wedge A.a_n = B.b_n} B$

Example

S1

sid	name	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	35.0

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

S1 ⋈ $S1.sid = R1.sid$ **R1**

Result

S1.Sid	sname	rating	age	R1.sid	bid	day
22	dustin	7	45.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

Equivalent to $\sigma_{S1.sid = R1.sid} (S1 \times R1)$

The most important and frequently used of the joins is the **INNER JOIN. They are also referred to as an **EQUIJOIN**.

Example

Student

SID	Name	Std ✓
101	Rohan	11
102	Mira	12

Subjects

Class	Subject
11	Maths
11	Physics
12	English
12	Chemistry

Student ⋈ *Student. Std = Subjects. Class* Subjects

Result

SID	Name	Std	Class	Subject
101	Rohan	11	11	Maths
101	Rohan	11	11	Physics
102	Mira	12	12	English
102	Mira	12	12	Chem

Some important note on Natural join

- **Natural join** can only be performed *if there is at least one common attribute* (column) that exist between two relations. In addition, the **attributes** must have the **same name and domain**.
- Natural join does not use any comparison operator.
- *It is same as equi join which occurs implicitly* by comparing all the **common attributes (columns)** in both relation, **but** difference is that **in Natural Join the common attributes appears only once.** The resulting schema will change.
- **Notation:** **$A \bowtie B$**
- The **result of the natural join** is the set of all combinations of tuples in two relations **A** and **B** that are **equal on their common attribute names.**

Outer Join

- An **inner join** includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use **outer joins** to include all the rest of the tuples from the participating relations in the resulting relation.
- The **outer join** operation is an **extension of the join operation** that **avoids loss of information**.
- **Outer Join** contains matching tuples that satisfy the matching condition, along with some or all tuples that do not satisfy the matching condition.
 - It is based on both matched or unmatched tuple.
 - It contains all rows from either one or both relations are present
- It uses **NULL** values.

Example

Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

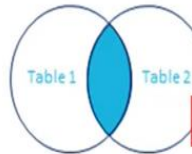
HOD

CID	Name
100	Rohan
102	Sara
104	Jiya

1st do the inner join:

Courses ⋈ HOD

CID	Course	Name
100	Database	Rohan
102	Electronics	Sara

Inner Join

- **Outer Join = Natural Join + Extra information**

(from left table, right table or both table)

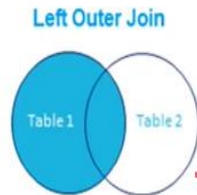
- There are **three kinds of outer joins**:

1. **Left outer join**
2. **Right outer join**
3. **Full outer join**

Left Outer Join

When applying **join** on two relations R1 and R2, some tuples of R1 or R2 does not appear in result set which does not satisfy the join conditions. **But..**

- ❑ In **Left outer join**, *all the tuples from the Left relation R1 are included in the resulting relation*. The tuples of R1 which do not satisfy join condition will have values as **NULL** for attributes of R2.
- ❑ In short:
 - ❑ All record from **left** table
 - ❑ Only matching records from right table
- ❑ **Symbol:** $\bowtie$
- ❑ **Notation:** **R1** $\bowtie$ **R2**

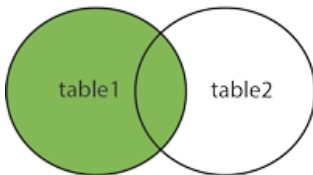


The LEFT JOIN keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

LEFT JOIN Syntax

```
SELECT column_name(s)  
FROM table1  
LEFT JOIN table2  
ON table1.column_name = table2.column_name;
```

LEFT JOIN



Example

Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

HOD

CID	Name
100	Rohan
102	Sara
104	Jiya

Courses ⋈ **HOD**

Result

CID	Course	Name
100	Database	Rohan
101	Mechanics	NULL
102	Electronics	Sara

Right Outer Join

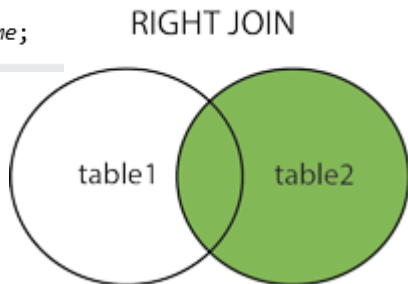
- In **Right outer join**, *all the tuples from the right relation R2 are included in the resulting relation*. The tuples of R2 which do not satisfy join condition will have values as **NULL** for attributes of R1.
- In short:
 - All record from **right** table
 - Only matching records from left table
- **Symbol:** $\bowtie$
- **Notation:** **R1** $\bowtie$ **R2**



The RIGHT JOIN keyword returns all records from the right table (table2), and the matching records from the left table (table1). The result is 0 records from the left side, if there is no match.

RIGHT JOIN Syntax

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```



Example

Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

HOD

CID	Name
100	Rohan
102	Sara
104	Jiya

Courses ⋈ HOD

CID	Course	Name
100	Database	Rohan
102	Electronics	Sara
104	NULL	Jiya

Full Outer Join

- In **Full outer join**, *all the tuples from both Left relation R1 and right relation R2 are included in the resulting relation*. The tuples of both relations R1 and R2 which do not satisfy join condition, their respective unmatched attributes are made **NULL**.
- In short:
 - All record from **all** table
- **Symbol:** $\bowtie$
- **Notation:** $R1 \bowtie R2$



Courses

CID	Course
100	Database
101	Mechanics
102	Electronics

HOD

CID	Name
100	Rohan
102	Sara
104	Jiya

Courses ⋈ HOD

CID	Course	Name
100	Database	Rohan
101	Mechanics	NULL
102	Electronics	Sara
104	NULL	Jiya

Thank You